

Sablier Bob / Escrow Audit Summary

Prepared by Cyfrin

Version 2.0

Lead Auditors

Dacian

MrPotatoMagic

March 25, 2026

Contents

1 About Cyfrin **2**

2 About This Document **2**

3 Disclaimer **2**

4 Copyright and License **2**

5 Audit Overview **3**

6 Audit Scope **3**

7 Risk Classification **4**

8 Protocol Summary **4**

8.1 Sablier Bob 4

8.2 Sablier Escrow 5

9 Attestation **5**

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Sablier on Bob / Escrow. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

It is deliberately abbreviated: the technical detail of individual findings is not reproduced here. The complete audit report, version 2.0 dated March 25, 2026, was delivered to Sablier, who retain it as the full record of the engagement. This summary was issued on October 29, 2026.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Sablier or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 5 days, the Cyfrin team conducted an audit on the Bob / Escrow code provided by Sablier. The findings consist of 1 Critical, 1 High, 4 Medium & 10 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	Bob / Escrow
Repository	lockup
Commit	9fbc34f8ac38...
Fix Commit	ffae95821335...
Audit Timeline	Feb 19th - Feb 25th, 2026
Methods	Manual Review, Formal Verification

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	1	1	0
High Risk	1	1	0
Medium Risk	4	3	1
Low Risk	10	4	6
Informational	9	3	6
Gas Optimizations	11	8	3
Total	36	20	16

6 Audit Scope

The audit scope was limited to:

```
bob/src/abstracts/SablierBobState.sol
bob/src/abstracts/SablierEscrowState.sol
bob/types/Bob.sol
bob/types/Escrow.sol
bob/src/BobVaultShare.sol
bob/src/SablierBob.sol
bob/src/SablierEscrow.sol
bob/src/SablierLidoAdapter.sol
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

8.1 Sablier Bob

Sablier Bob is a price-gated vault protocol for conditional token releases with optional yield generation. It uses a singleton architecture where all vaults are managed in a single `SablierBob` contract. Each vault deploys a separate `BobVaultShare` ERC-20 token that tracks user deposits at a 1:1 ratio.

8.1.1 Vault Lifecycle

Users deposit ERC-20 tokens into vaults via `enter` and receive share tokens. Vaults have three statuses computed at runtime from stored state:

- **ACTIVE**: Accepting deposits. `lastSyncedPrice < targetPrice` and `block.timestamp < expiry`
- **SETTLED**: Synced price reached target. `lastSyncedPrice >= targetPrice`
- **EXPIRED**: Expiry timestamp passed. `block.timestamp >= expiry` (checked before SETTLED)

`lastSyncedPrice` is updated from the Chainlink oracle via `syncPriceFromOracle`, `enter` or `redeem`.

8.1.2 Redemption

After settlement or expiry, share holders call `redeem` to burn their shares and receive tokens:

- **Non-adapter vaults**: The caller pays a native token fee via `msg.value`. Tokens are transferred 1:1 for shares burned
- **Adapter vaults**: The first redemption triggers `_unstakeFullAmountViaAdapter`, which converts all yield-bearing tokens back to the deposit token. A yield fee (max 20%, snapshotted at vault creation) is deducted only on positive yield. `unstakeTokensViaAdapter` can also trigger this conversion before individual redemptions

8.1.3 Lido Adapter

The optional `SablierLidoAdapter` generates yield by staking deposited tokens through the flow: WETH $\rightarrow$ ETH $\rightarrow$ stETH (Lido) $\rightarrow$ wstETH. The adapter tracks per-user wstETH balances and rebalances proportionally on share transfers via the `onShareTransfer` hook.

Unstaking supports two mutually exclusive exit paths per vault:

- **Curve swap**: wstETH $\rightarrow$ stETH $\rightarrow$ ETH (Curve pool exchange) $\rightarrow$ WETH, with a configurable slippage tolerance (max 5%)
- **Lido native withdrawal**: `requestLidoWithdrawal` unwraps wstETH $\rightarrow$ stETH and submits withdrawal requests to the Lido withdrawal queue. Once finalized, `unstakeFullAmount` claims the withdrawals and wraps the received ETH $\rightarrow$ WETH

Individual user redemptions are processed via `processRedemption`, which distributes WETH proportionally based on each user's wstETH share of the vault total.

8.1.4 Key Constants and Limits

Parameter	Value
Max yield fee (adapter)	20%
Max slippage tolerance (Curve)	5%
Oracle decimals	8
Amount precision	`uint128` for all monetary values

8.2 Sablier Escrow

Sablier Escrow is an over-the-counter (OTC) token swap protocol that enables peer-to-peer ERC-20 token swaps. It uses a singleton architecture where all orders are managed in a single `SablierEscrow` contract.

8.2.1 Order Lifecycle

Sellers create orders by depositing a sell token and specifying a buy token, minimum buy amount, optional buyer, and optional expiry. Orders have four statuses computed at runtime:

- **OPEN**: Available to be filled. Not filled, not canceled, and not expired
- **FILLED**: A buyer has filled the order
- **CANCELLED**: The seller canceled the order
- **EXPIRED**: The expiry timestamp has passed (`expiryTime > 0 && block.timestamp >= expiryTime`)

Setting `buyer = address(0)` creates a public order anyone can fill. Setting a specific buyer creates a private order. Setting `expiryTime = 0` creates an order that never expires. Buyers can pay more than `minBuyAmount` for price improvement.

8.2.2 Fee Structure

A configurable trade fee (max 2%) is deducted bilaterally on fill:

- From the buyer's received sell tokens: `sellAmount × tradeFee`
- From the seller's received buy tokens: `buyAmount × tradeFee`

Both fee portions are sent to the comptroller contract.

9 Attestation

Cyfrin attests that it performed a security review of the Bob / Escrow code provided by Sablier.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-03-25-cyfrin-sablier-bob-escrow-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Sablier has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=bb2957d7d175295ec9729f63dc3a35c40709a8e7c94400d4d9c7a85d3c5cef5e
F=2026-03-25-cyfrin-sablier-bob-escrow-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.